

Structured Output in LangChain

1. Introduction:

When we use Large Language Models (LLMs) like ChatGPT, they usually respond with **plain text**.

But in real-world applications (apps, websites, automation), we often need **organized data**, not just text.

👉 That's where **Structured Output** comes in.

2. Recap

Before this topic, you likely learned:

- How to send prompts to LLMs
- How LLMs generate human-like text

But:

❌ Text is hard for machines to process

✅ Structured data is easy for machines to use

3. What is Structured Output?

✅ **Simple Definition:**

Structured output means the LLM returns data in a **fixed, organized format** (like JSON), instead of free text.

Example

Unstructured Output (Normal LLM response):

John is 25 years old and lives in Pakistan.

Structured Output (JSON format):

```
{  
  "name": "John",  
  "age": 25,  
  "country": "Pakistan"  
}
```

💡 Why this matters:

- Machines understand JSON easily
 - No need to “guess” information from text
-

4. Why Do We Need Structured Output?

Problem with Unstructured Text:

- Hard to extract data
- Error-prone parsing
- Not reliable for automation

Benefits of Structured Output:

1. Easy Data Processing

- You can directly access fields like name, age

2. Automation Friendly

- Works well with APIs, databases, dashboards

3. Consistency

- Always same format → fewer bugs

4. Integration

- Connect LLMs with:
 - Databases
 - APIs
 - Backend systems

Real-world Example:

Imagine building a **job application system**

Without structured output:

- You must manually extract name, skills from text

With structured output:

```
{  
  "name": "Ali",  
  "skills": ["Python", "SQL"],  
  "experience": 2  
}
```

 Directly store in database ✓

05. Ways to Get Structured Output

There are multiple ways in LangChain:

1. Prompt Engineering

Tell the LLM:

"Respond only in JSON format"

⚠️ Not fully reliable

2. Built-in Structured Output Tools (Best Way)

LangChain provides tools to enforce structure:

- with_structured_output()
 - TypedDict
 - Pydantic
-

6. with_structured_output Function

✅ **What it does:**

It forces the LLM to return output in a **specific structure**

Example (Conceptual):

```
model.with_structured_output(schema)
```

👉 You define a schema → LLM follows it

💡 **Think of it like:**

You are giving the AI a **form to fill**, instead of letting it write freely.

7. TypedDict

✅ **What is it?**

A Python way to define the **structure of a dictionary**

Example:

```
from typing import TypedDict
```

```
class Person(TypedDict):
```

```
    name: str
```

```
    age: int
```

💡 **Meaning:**

- name must be string
 - age must be integer
-

✅ **Why use it?**

- Simple
 - Lightweight
 - Good for basic structures
-

8. Pydantic

What is it?

A powerful Python library for **data validation and structure**

Example:

```
from pydantic import BaseModel
```

```
class Person(BaseModel):
```

```
    name: str
```

```
    age: int
```

Why Pydantic is better:

- Validates data automatically
 - Gives errors if data is wrong
 - Supports complex structures
-

Example:

If LLM returns:

```
{ "name": "Ali", "age": "twenty" }
```

 Pydantic will reject it 

Best for:

- Production systems
 - APIs
 - Complex applications
-

9. JSON

What is JSON?

A standard format for structured data

Example:

```
{
```

```
"product": "Laptop",  
"price": 1000  
}
```

Why JSON?

- Universal format
 - Used in APIs
 - Easy to read + process
-

10. When to Use What?

Use TypedDict:

- Simple projects
 - Basic structure
 - Learning stage
-

Use Pydantic:

- Real applications
 - Need validation
 - Complex schemas
-

Use Raw JSON prompting:

- Quick experiments
 - Not critical systems
-

11. Important Points

Key Takeaways:

1. LLMs naturally generate text → not structured
 2. Structured output is essential for real-world apps
 3. Always prefer:
 - with_structured_output
 - Pydantic (for serious projects)
 4. Avoid relying only on prompts
 5. Validation is critical (use Pydantic)
-

12. Outro (Final Understanding)

Big Picture:

Structured Output turns LLMs from:

✗ Chatbots (text generators)

➡ Into:

✓ **Smart data processors**

🔥 Final Real-World Flow Example

1. User input:

"I am Ali, 22 years old, I know Python"

2. LLM (Structured Output):

```
{  
  "name": "Ali",  
  "age": 22,  
  "skills": ["Python"]  
}
```

3. System:

- Save to database
 - Send to API
 - Show in dashboard
-

✓ Quick Summary

- **Structured Output = Organized Data (JSON)**
 - Makes LLMs usable in real applications
 - Tools in LangChain:
 - with_structured_output
 - TypedDict
 - Pydantic
 - Best practice → Use **Pydantic + structured output**
-